Lab 3 Report

Simulation of a RISC-V based SoC using GVSoC

Digital Computing Systems (CESE4130) · 2026/27

Part	Result
1 – LWI instruction	Implemented; check prints “LWI implemented correctly”. 277,398 → 268,182 cycles (−9,216).
2 – 16-core SoC	16 cores + global memory; parallel count of 3s. **make verify: all 48 tests passed.**
Profiling	Array-1: 35,164 → 5,790 cycles (6.07× on 16 cores); ≈ 90% parallelizable. Array-3: best on 4 cores.
Bonus – histogram	Correct on 1–16 cores (500/500/600/1000/1000/100/100); 272,198 → 46,171 cycles.

Part 1 – Load Word Indexed (LWI)

1a. Baseline

In `img-blur-baseline`, `make gvsoc all count-cycles` gives **277,398 cycles** for the image-blur kernel with the original ISA.

1b. Adding the instruction

Encoding (R-type): funct7 = `0000000`, funct3 = `010`, opcode = `0101011` (custom-1). Semantics: **rd = mem[rs1 + rs2]** (rs1 = base, rs2 = index).

Decoder – `core/models/cpu/iss/isa_gen/isa_riscv_gen.py`, class `Rv32i`, next to `lw`, without a fast handler:

```
Instr('lwi',    Format_R,    '0000000 ----- ----- 010 ----- 0101011', tags=["load"]),
```

It has two source registers like `add`, so it uses `Format_R` (rd = output 0, rs1 = input 0, rs2 = input 1), not the L format whose second operand is an immediate. The `load` tag gives it the timing of a load. Opcode 0101011 is only used by the CORE-V / PULP extensions, which an `rv64imafdc` core does not include.

Execution – `core/models/cpu/iss/include/isa/rv32i.hpp`, after `lw_exec`: the `lw` load path, with the address computed as `add` would:

```
// Load Word Indexed: like lw, but the address is the sum of the base (rs1) and
// index (rs2) registers, as add would compute it, instead of base + immediate
static inline iss_reg_t lwi_exec(Iss *iss, iss_insn_t *insn, iss_reg_t pc)
{
    // If address register is not valid, we are accessing random location, trigger
    // a memcheck fail
    iss->regfile.memcheck_access_reg(REG_IN(0));
    iss->regfile.memcheck_access_reg(REG_IN(1));

    iss_addr_t addr = REG_GET(0) + REG_GET(1);
    iss->lsu.stack_access_check(REG_IN(0), addr);
    if (iss->lsu.load_signed_perf<int32_t>(insn, addr, 4, REG_OUT(0)))
    {
        return pc;
    }
    return iss_insn_next(iss, insn, pc);
}
```

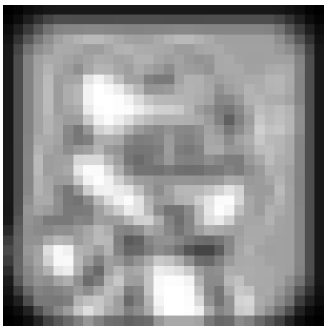
Assembly: the provided `.s` files already contain the instruction as `.word 0x00F727AB` = rd a5, rs1 a4 (base), rs2 a5 (index), 9 times per output pixel. It replaces `add a5,a4,a5` + `lw a5,0(a5)`; the index was already scaled by 4 by the preceding `slli`.

1c. Verification and modified-ISA run

`check-image-blur-modified`: `make gvsoc all run image` prints **“LWI implemented correctly”** and produces the blurred image below. `img-blur-modified`: `make gvsoc all count-cycles` gives **268,182 cycles**.



Input image



Output of the LWI check program (blurred)

Version	Cycles	Δ vs baseline
Original ISA (baseline)	277,398	–
Modified ISA (lwi)	268,182	−9,216 (-3.3%)

Question – difference in cycles?

277,398 − 268,182 = **9,216 cycles** (≈3.3% fewer). The image has 32 × 32 = 1,024 output pixels and the 3×3 blur does 9 loads per pixel: 1,024 × 9 = 9,216. Each `lwi` replaces an `add` + `lw` pair, so every load saves exactly one instruction, one cycle. The memory access itself is unchanged, and most of the loop is still index arithmetic, so the overall gain is modest.

Part 2 – Multi-core RISC-V SoC

2a. Baseline

With the template (1 core, Array-1), `make gvsoc all run` prints `The total 3s are:56`.

2b/2c. Adding 16 cores and the global memory – my_system.py

```
# Global memory shared by all the cores: sync register at offset 0,
# then one counter per core
global_memory = global_mem.Global_Mem(self, 'global_mem')
ico.o_MAP(global_memory.i_INPUT(), 'global_mem', base=0x20000000, size=0x00001000, rm_base=True)

# Instantiates the cores and connect fetch and data to the interconnect.
# Core 0 keeps the name 'host', the plot scripts trace 'host/insn'.
cores = []
for core_id in range(NB_CORES):
    name = 'host' if core_id == 0 else 'core%d' % core_id
    core = cpu.iss.riscv.Riscv(self, name, isa='rv64imafdc', core_id = core_id)
    core.o_FETCH     ( ico.i_INPUT     ())
    core.o_DATA      ( ico.i_INPUT     ())
    core.o_DATA_DEBUG(ico.i_INPUT    ())
    cores.append(core)


# Finally connect an ELF loader, which will execute first and will then
# send to every core the boot address and notify them they can start
loader = utils.loader.loader.ElfLoader(self, 'loader', binary=binary)
loader.o_OUT     ( ico.i_INPUT     ())
for core in cores:
    loader.o_START   ( core.i_FETCHEN  ())
    loader.o_ENTRY   ( core.i_ENTRY    ())
```

Core 0 keeps the name `host` because the profiling scripts trace `host/insn`. Every core gets a distinct `core_id` (its `mhartid`), its fetch/data/debug ports on the interconnect, and is started by the same loader. `Global_Mem` is mapped at **0x20000000**, size **0x1000**; `rm_base` makes the component see offsets: 0x0 = `sync_reg`, 0x4 + 4·i = `count[i]`, all initialised to −1. `crt0.S` already gives each hart its own stack.

2d. Parallelizing main.c

- Unused cores (core_id ≥ NUM_CORES) write 0 to their counter and pause.
- Each core counts a contiguous slice of ceil(arr_size / NUM_CORES) elements; the last slices may be shorter or empty, and the index check stops at the end of the array.
- The template read `arr[start_index + i]` although `i` already starts at `start_index`; it now reads `arr[i]`. Its “protection” `if (arr[i] > arr_size)` compared an element value with the size; it is replaced by a real bound on the index.
- Each core stores its count; cores 1–15 pause; core 0 waits in `sync_cores()` until no counter is −1, sums, prints and writes `sync = 0` to release the others.

```
if (core_id >= NUM_CORES){
    global_mem[core_id] = 0;
    pause_core();
}

// Slice the array into NUM_CORES contiguous sections. The slice size is
// rounded up so every element is covered; the last slices may be shorter
// or empty when arr_size isn't a multiple of NUM_CORES.
int arr_size = sizeof(arr) / sizeof(arr[0]);
int arr_size_per_core = (arr_size + NUM_CORES - 1) / NUM_CORES;
int start_index = core_id * arr_size_per_core;
int end_index = start_index + arr_size_per_core;
int local_counter = 0;

// Only iterate over the array slice assigned to the core
for(int i = start_index; i < end_index; i++){
    // Protection from Illegal Memory Access
    if(i >= arr_size){
        break;
    }

    // Count the number of 3s in the array and save in local reg
    if(arr[i] == 3){
        local_counter++;
    }
}

// Update the global variable
global_mem[core_id] = local_counter;

// Pause all cores except Core 0
if(core_id != 0){
    pause_core();
}

// Synchronize all the cores
sync_cores();

// Calculate the total number of 3s
int total_3s = 0;
for (int i = 0; i < NUM_CORES; i++){
    total_3s += global_mem[i];
}

// Print Final Value
printf("The total 3s are:%d\n", total_3s);

// Release the paused cores so that all the cores can terminate
*sync = 0;
```

Verification: `make gvsoc verify` → **“All tests were passed!!”** (3 arrays × 1–16 cores = 48 configurations).

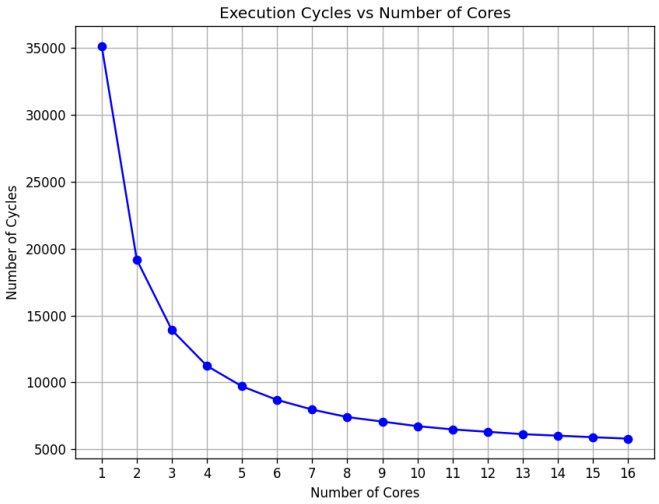
Verification & Profiling

Measured cycles (core 0, first to last instruction in main):

Cores	Array-1	Speedup	Array-2	Δ (1−2)	Array-3
1	35,164	1.00×	34,847	317	4,224
2	19,177	1.83×	18,904	273	3,693
3	13,914	2.53×	13,671	243	3,576
4	11,234	3.13×	10,999	235	3,494
5	9,701	3.62×	9,476	225	3,537
6	8,688	4.05×	8,467	221	3,520
7	7,965	4.41×	7,746	219	3,567
8	7,406	4.75×	7,187	219	3,550
9	7,066	4.98×	6,847	219	3,594
10	6,725	5.23×	6,510	215	3,641
11	6,482	5.42×	6,269	213	3,688
12	6,303	5.58×	6,092	211	3,703
13	6,126	5.74×	5,915	211	3,750
14	6,013	5.85×	5,802	211	3,797
15	5,900	5.96×	5,689	211	3,844
16	5,790	6.07×	5,579	211	3,862

Highlighted row: 4 cores = minimum for Array-3.

Array-1 (make plot)



Execution cycles vs number of cores – Array-1

Q1. Why is the graph non-linear?

Only the counting loop is divided between the cores, so only that part shrinks as 1/N. The rest is serial on core 0: start-up, the final sum and above all the `printf`, which alone takes 2,885 cycles (measured in the trace). That is 8% of the 1-core time but half of the 16-core time. Some overhead even grows with N: `sync_cores()` and the final sum loop over NUM_CORES (core 0 spends 91 cycles in sync on 1 core, 800 on 16), and core 0 has to wait for the slowest slice. So $T(N) \approx T_{serial} + T_{parallel}/N + c \cdot N$: a hyperbola that flattens out instead of a straight line.

Q2. Amdahl's law – parallel fraction p

Speedup at 16 cores S = 35,164 / 5,790 = 6.07. Solving $1/S = (1 - p) + p/16$:

```
p = (1 − 1/S) / (1 − 1/16) = (1 − 0.165) / 0.9375 ≈ 0.891
```

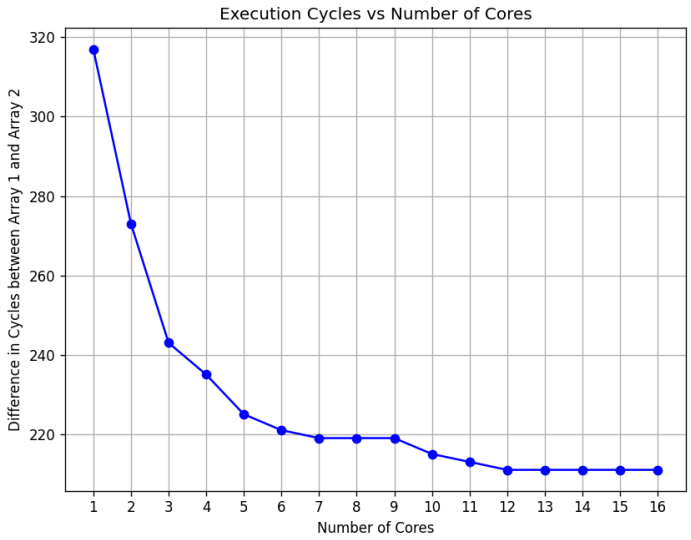
Cores used	Speedup S	p
2	1.83	0.909
4	3.13	0.907
8	4.75	0.902
16	6.07	0.891
Fit over all 16 points	–	0.897

> **≈ 90% of the program is parallelizable** (0.89–0.91 depending on the points used). The trace agrees: on 1 core the counting phase is 32,188 of 35,164 cycles (≈92%). The estimate falls slightly with N because the synchronisation overhead grows with N, which Amdahl's law does not model.

Q3. Trend beyond 16 cores?

The curve keeps flattening towards the serial floor (≈ 3,600 cycles from the fit); the speedup is capped at $1/(1 - p) \approx 9.7\times$. Before reaching it, each core gets almost no work (1000/N elements) while the sync and sum loops keep growing with N, so the curve reaches a minimum and rises again, as Array-3 already does. The hardware would also need more than the 16 counters in the global memory.

Array-1 vs Array-2 (make diff)



Cycle difference Array-1 − Array-2 vs number of cores

Q1. Difference in execution between Array-1 and Array-2?

Same size, same loop. For every element equal to 3, Array-1 executes the extra `local_counter++` (lw / addiw / sw) instead of branching past it; Array-2 has no 3s. The final `printf` also prints “56” instead of “0”, one more digit to convert. Splitting core 0's trace shows both parts:

	Counting phase	printf → end	Total Δ
1 core	+112 (56 threes × 2 cycles)	+205	317
16 cores	+6 (core 0: 3 threes; busiest slice: 7)	+205	211

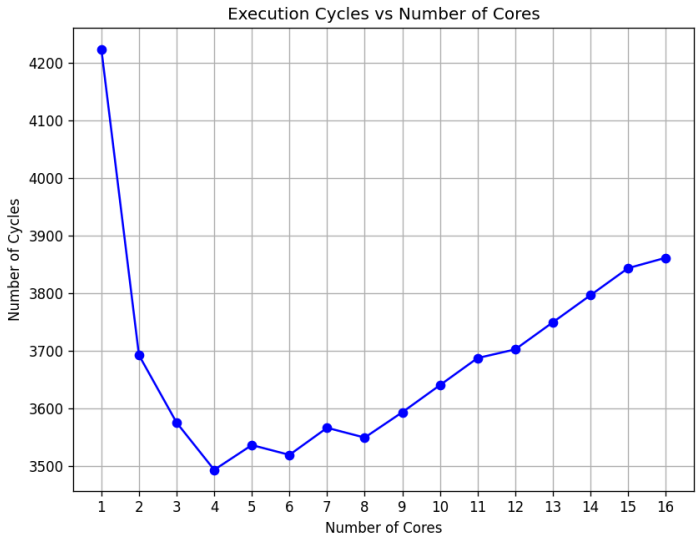
Q2. Why is the difference decreasing? Will it reach zero?

The 3s are spread over the cores, which count in parallel, and core 0 only waits for the slowest one. So only the 3s in the busiest slice are on the critical path: 56 on 1 core, 34 on 2, 19 on 4, 11 on 8, 7 on 16. **It will not reach zero**: it levels off at about 211 cycles from 12 cores onward. 205 of those come from the `printf`, which does not depend on the core count, and a slice that still contains 3s keeps the rest above zero.

Q3. What if Array-1 had more 3s?

Each extra 3 in the busiest slice costs about 2 cycles. On 1 core the difference grows linearly with the number of 3s; with N cores each extra 3 adds only ≈1/N of that, so the curve starts higher but still falls towards the `printf` floor. A count with one more digit (≥ 100) raises that floor by roughly another 200 cycles, and clustered 3s (load imbalance) make the difference shrink more slowly than evenly spread ones.

Array-3 (make plot)



Execution cycles vs number of cores – Array-3 (36 elements)

Q1. Why is the trend different?

With only 36 elements the divisible work is about 1,100 cycles, small next to the fixed serial part and the ≈40 cycles of sync and summing added per extra core. Up to 4 cores the counting shrinks faster than the overhead grows; after that each added core saves 2–3 elements but still costs its sync and sum, so the curve rises roughly linearly. The zigzag comes from 36 not dividing evenly by N.

Q2. Ideal number of cores for Array-3: 4

Minimum of 3,494 cycles (9 elements per core), a speedup of only 1.21×; 5–6 cores are within about 1%. With $T(N) \approx T_S + W/N + c \cdot N$, $W \approx 1{,}100$ and $c \approx 40$, the optimum $N = \sqrt{(W/c)} \approx 5$ agrees.

Bonus – Character histogram

- Copied `Part2/my_system.py` to `bonus/my_system.py` (the bonus global memory holds 16 × 7 = 112 counters).
- Each core takes ceil(arr_size / NUM_CORES) characters (CUDA-grid style); the index check handles the shorter last slice.
- Each core writes its 7 bins to `global_mem[core_id*NUM_BINS + bin]`; unused cores clear only their own 7 words (`global_mem[core_id]` would land in a used core's bins) and pause.
- Core 0 synchronises over NUM_CORES·NUM_BINS words, sums every bin over the used cores and prints the histogram. The big test array is active, as required for sign-off.

```
int arr_size_per_core = (arr_size + NUM_CORES - 1) / NUM_CORES;
int start_index = core_id * arr_size_per_core;
int end_index = start_index + arr_size_per_core;
int local_counter[7] = {0, 0, 0, 0, 0, 0, 0};

int bin_index;
for (int i = start_index; i < end_index; i++) {
    // Protection from illegal memory access
    if (i >= arr_size) {
        break;
    }
    char c = arr[i];
    if (c >= 'a' && c <= 'z') {
        bin_index = (c - 'a') / N_LETTERS_PER_BIN;
        local_counter[bin_index]++;
    }
}

// Update of the global variable
for (int i = 0; i < NUM_BINS; i++) {
    global_mem[core_id * NUM_BINS + i] = local_counter[i];
}

// Pause all cores except Core 0
if(core_id != 0){
    pause_core();
}

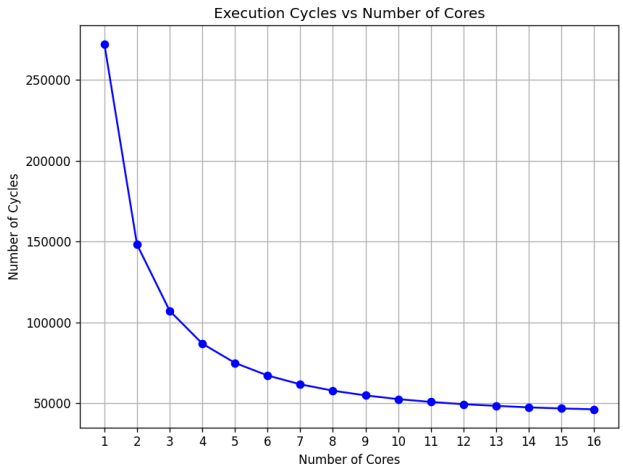
// Synchronize all the cores
sync_cores();

// Calculate the final value of each bin by adding up the partial
// histograms of all the used cores
int final_count[7] = {0, 0, 0, 0, 0, 0, 0};
for (int i = 0; i < NUM_CORES; i++) {
    for (int j = 0; j < NUM_BINS; j++) {
        final_count[j] += global_mem[i * NUM_BINS + j];
    }
}

// Print Final Value
printf("Output Data:\n");
```

Bin	0	1	2	3	4	5	6
Output (1–16 cores)	500	500	600	1000	1000	100	100

Checked for every core count from 1 to 16; all match the expected output.



Bonus – execution cycles vs number of cores (272,198 → 46,171, 5.9× on 16 cores)

Files changed

File	Change
core/models/cpu/iss/isa_gen/isa_riscv_gen.py	lwi decoder entry in Rv32i
core/models/cpu/iss/include/isa/rv32i.hpp	lwi_exec
cese4130-lab4/Part2/my_system.py	16 cores + global memory
cese4130-lab4/Part2/main.c	Parallel count of 3s
cese4130-lab4/bonus/my_system.py	Copy of Part2 system
cese4130-lab4/bonus/main.c	Parallel histogram, big array active